

# awesome 视频处理IP

## 开发课题说明书

基于易灵思 TJ180 FPGA 的模块化电路系统  
图像信号处理（ISP）算法硬件实现 & Vibe-Coding 开发模式

版本: V1.0.0 | 日期: 2026-04-15 | 状态: 发布

课题编号: AWESOM-IP-2026-Q1

| 属性   | 值                 |
|------|-------------------|
| 版本   | 1.0.0             |
| 日期   | 2026-04-15        |
| 状态   | 发布                |
| 课题编号 | AWESOM-IP-2026-Q1 |

## 一、课题概述

### 1.1 课题背景

awesome 是一款基于易灵思 TJ180 FPGA 的模块化电路系统，核心板集成了 FPGA 芯片、2Gb LPDDR4X 存储和丰富的接口资源。本课题旨在让学生通过实际项目开发，掌握 FPGA 视频处理 IP 的设计方法，理解图像信号处理（ISP）算法的硬件实现，并积累完整的 IP 开发经验。

本课题采用 **Vibe-Coding** 开发模式，即在 AI 辅助下完成代码编写和调试。学生将体验从需求分析、架构设计、代码实现、仿真验证到板级测试的完整开发流程。

### 1.2 课题目标

- 理解 AXI-Stream 视频接口标准，能够设计和实现符合标准的视频处理模块
- 掌握图像信号处理（ISP）算法的基本原理和硬件实现方法
- 熟悉 FPGA 开发流程，包括仿真、综合、实现和板级验证
- 学会使用 AI 辅助工具（Vibe-Coding）提升开发效率
- 积累模块化、可复用 IP 的设计经验

### 1.3 开发环境

| 组件     | 规格要求                                  |
|--------|---------------------------------------|
| FPGA器件 | 易灵思 TJ180A484S                        |
| 开发工具   | Efinity 2024.2 或更高版本                  |
| 仿真工具   | ModelSim / Verilator / Icarus Verilog |
| 硬件平台   | awesome 核心板                           |

| 组件   | 规格要求    |
|------|---------|
| 编程语言 | Verilog |

1.4 时间安排

| 阶段        | 时间   | 任务内容               |
|-----------|------|--------------------|
| Phase 1.1 | 需求分析 | 阅读文档，理解 IP 功能和接口要求 |
| Phase 1.2 | 架构设计 | 确定算法方案，设计模块结构      |
| Phase 1.3 | 代码开发 | 实现 IP 核心逻辑，AI 辅助编程 |
| Phase 1.4 | 仿真验证 | 功能仿真和时序仿真          |
| Phase 1.5 | 综合实现 | 综合、实现、生成比特流        |
| Phase 2.1 | 板级验证 | 比特流下载，板级测试验证       |
| Phase 2.2 | 文档整理 | 编写技术文档和用户手册        |

二、公共规格定义

2.1 系统参数

本课题所有 IP 开发均基于以下系统参数：

| 参数类别 | 参数名称       | 规格值         | 说明       |
|------|------------|-------------|----------|
| 分辨率  | MAX_WIDTH  | 3840        | 最大像素宽度   |
| 分辨率  | MAX_HEIGHT | 2160        | 最大像素高度   |
| 像素精度 | DATA_WIDTH | 8 / 10 / 12 | 可配置位宽    |
| 时钟频率 | CLK_FREQ   | 125 MHz     | 系统工作时钟   |
| 并行度  | PARALLEL   | 4           | 4 像素并行处理 |

## 2.2 时钟架构

| 时钟域       | 频率       | 用途         | 来源         |
|-----------|----------|------------|------------|
| sys_clk   | 125 MHz  | 主系统时钟      | 核心板晶振      |
| pixel_clk | 125 MHz  | 像素处理时钟     | sys_clk 分频 |
| ddr_clk   | 1500 MHz | LPDDR4X 接口 | FPGA PLL   |

### 时钟约束：

```
TCL
# 系统时钟约束
create_clock -name sys_clk -period 8.0 [get_ports sys_clk]

# Pixel 时钟约束
create_clock -name pixel_clk -period 8.0 -waveform {0 4.0} [get_pixels pixel_clk]
```

## 2.3 AXI-Stream 视频接口标准

所有视频处理 IP 采用统一的 AXI-Stream 接口协议。

### 2.3.1 接口信号定义

| 信号名           | 方向 | 位宽 | 说明         |
|---------------|----|----|------------|
| aclk          | 输入 | 1  | 时钟信号       |
| aresetn       | 输入 | 1  | 异步低有效复位    |
| s_axis_tvalid | 输入 | 1  | 发送方数据有效    |
| s_axis_tready | 输出 | 1  | 接收方就绪      |
| s_axis_tdata  | 输入 | 可变 | 视频数据       |
| s_axis_tuser  | 输入 | 1  | 帧起始标志（SOF） |
| s_axis_tlast  | 输入 | 1  | 行结束标志（EOL） |
| s_axis_tkeep  | 输入 | 可变 | 字节有效掩码     |
| m_axis_tvalid | 输出 | 1  | 发送方数据有效    |
| m_axis_tready | 输入 | 1  | 接收方就绪      |

| 信号名          | 方向 | 位宽 | 说明     |
|--------------|----|----|--------|
| m_axis_tdata | 输出 | 可变 | 视频数据   |
| m_axis_tuser | 输出 | 1  | 帧起始标志  |
| m_axis_tlast | 输出 | 1  | 行结束标志  |
| m_axis_tkeep | 输出 | 可变 | 字节有效掩码 |

2.3.2 时序图

图 2-1: AXI-Stream 视频接口握手时序示意

2.3.3 数据格式

| 视频格式   | 每像素位宽   | tdata 宽度 | 数据排列                           |
|--------|---------|----------|--------------------------------|
| RAW8   | 8 bits  | 32 bits  | {24'd0, pixel[7:0]}            |
| RAW10  | 10 bits | 32 bits  | {22'd0, pixel[9:0]}            |
| RAW12  | 12 bits | 32 bits  | {20'd0, pixel[11:0]}           |
| RGB888 | 24 bits | 32 bits  | {8'd0, r[7:0], g[7:0], b[7:0]} |
| YUV422 | 16 bits | 32 bits  | {8'd0, y[7:0], u[7:0], v[7:0]} |

2.4 寄存器接口标准

每个 IP 核提供标准的寄存器接口，用于配置和控制。

2.4.1 寄存器空间映射

| 地址偏移 | 寄存器名       | 访问 | 说明     |
|------|------------|----|--------|
| 0x00 | IP_VERSION | RO | IP 版本号 |
| 0x04 | IP_CTRL    | RW | 控制寄存器  |
| 0x08 | IP_STATUS  | RO | 状态寄存器  |
| 0x0C | IP_RESET   | WO | 复位寄存器  |
| 0x10 | IMG_WIDTH  | RW | 图像宽度   |
| 0x14 | IMG_HEIGHT | RW | 图像高度   |

| 地址偏移        | 寄存器名       | 访问 | 说明   |
|-------------|------------|----|------|
| 0x18        | IMG_FORMAT | RW | 图像格式 |
| 0x1C - 0x3C | IP_PARAM_* | RW | 算法参数 |

## 2.4.2 寄存器定义示例

```

Verilog
// IP 版本寄存器 (0x00) - 只读
localparam REG_IP_VERSION = 8'h00;
// [31:16] - 主版本号
// [15:8] - 次版本号
// [7:0] - 修订号

// IP 控制寄存器 (0x04) - 读写
localparam REG_IP_CTRL = 8'h04;
// [0] - 模块使能
// [1] - 旁路模式
// [2] - 调试模式

// 图像宽度寄存器 (0x10) - 读写
localparam REG_IMG_WIDTH = 8'h10;
// [15:0] - 图像宽度像素数

```

## 2.5 IP 核基本结构

所有视频处理 IP 遵循统一的模块结构：

```

Verilog
module ip_core_name #(
    // 参数定义
    parameter C_DATA_WIDTH = 24,
    parameter C_MAX_WIDTH = 3840,
    parameter C_MAX_HEIGHT = 2160
) (
    // 系统接口
    input wire      aclk,
    input wire      aresetn,

    // 配置接口
    input wire [7:0] cfg_addr,
    input wire [31:0] cfg_wdata,
    input wire      cfg_wen,
    input wire      cfg_ren,
    output wire [31:0] cfg_rdata,

    // AXI-Stream 输入
    input wire      s_axis_tvalid,
    output wire     s_axis_tready,
    input wire [C_DATA_WIDTH-1:0] s_axis_tdata,
    input wire      s_axis_tuser,
    input wire      s_axis_tlast,

```

```
// AXI-Stream 输出
output wire      m_axis_tvalid,
input  wire      m_axis_tready,
output wire [C_DATA_WIDTH-1:0] m_axis_tdata,
output wire      m_axis_tuser,
output wire      m_axis_tlast
);
```

2.6 BRAM 资源使用约束

| 资源类型        | 可用量    | 说明            |
|-------------|--------|---------------|
| BRAM 总数     | 1280 块 | TJ180 总资源     |
| Pipeline 使用 | ~50 块  | 参考设计 Pipeline |
| IP 单模块限制    | ≤ 16 块 | 建议每个 IP 不超过   |

行缓冲计算方法：

单行数据量 = 图像宽度 × 每像素字节数  
所需 BRAM 块数 = ceil(单行数据量 / 10Kb)

示例：RGB888 格式，1920 像素宽度

单行数据量 = 1920 × 3 = 5760 bytes = 46.08 Kbits  
所需 BRAM 块数 = ceil(46.08 / 10) = 5 块

2.7 性能指标要求

| 指标    | 要求          | 说明         |
|-------|-------------|------------|
| 最高分辨率 | 3840 × 2160 | 4K 分辨率     |
| 最高帧率  | 60 fps      | 4K @ 60fps |
| 流水线深度 | ≤ 16 级      | 控制最大延迟     |
| 吞吐量   | 1 像素 / 时钟   | 实时处理       |
| 延迟    | ≤ 1 行       | 可配置 BYPASS |

### 三、开发要求

#### 3.1 Vibe-Coding 开发流程

本课题采用 Vibe-Coding 模式，即 AI 辅助编程开发。

##### 3.1.1 提示词工程规范

开发过程中需编写结构化的提示词，模板如下：

提示词模板

# 角色

你是一位专业的 FPGA 视频处理工程师。

# 任务

实现 [IP核名称]，功能为 [功能描述]。

# 输入输出规格

- 输入: AXI-Stream 视频流，格式 [格式]
- 输出: AXI-Stream 视频流，格式 [格式]
- 延迟要求: [延迟要求]

# 技术约束

- 语言: Verilog
- 时钟频率: 125 MHz
- 并行度: 4 像素 / 时钟
- BRAM 使用:  $\leq$  [N] 块

# 算法要求

[详细算法描述]

# 接口要求

- 遵循 awesom IP 接口标准
- 包含寄存器配置接口
- 支持 BYPASS 模式

# 输出要求

1. 完整的模块代码
2. 关键算法的注释说明
3. 测试激励代码

##### 3.1.2 AI 辅助开发阶段

| 阶段   | AI 辅助内容       | 学生职责    |
|------|---------------|---------|
| 架构设计 | 提供架构建议        | 审核和决策   |
| 代码编写 | 生成 Verilog 代码 | 审核和修改   |
| 仿真调试 | 提供调试建议        | 定位和解决问题 |

| 阶段   | AI 辅助内容 | 学生职责  |
|------|---------|-------|
| 优化改进 | 资源优化方案  | 评估和实施 |

### 3.2 代码规范

#### 3.2.1 命名规范

| 元素   | 规范     | 示例               |
|------|--------|------------------|
| 模块名  | 小写下划线  | color_space_conv |
| 信号名  | 小写下划线  | video_tvalid     |
| 参数名  | 大写下划线  | C_MAX_WIDTH      |
| 寄存器名 | 全大写下划线 | REG_IP_CTRL      |
| 常量   | 全大写下划线 | MAX_WIDTH        |

#### 3.2.2 编码风格

```
Verilog
// 模块定义
module module_name #(
    parameter C_PARAM = 8
) (
    // 接口定义
    input wire      aclk,
    input wire      aresetn,

    // 输入输出端口分组
    // 系统接口
    input wire      cfg_clk,

    // 数据接口
    input wire [7:0] data_in,
    output wire [7:0] data_out
);

// 中间信号定义
reg [7:0] data_delay;
wire      process_en;

// 组合逻辑
assign data_out = data_in + 1;

// 时序逻辑
always @(posedge aclk) begin
    if (!aresetn) begin
        data_delay <= 8'd0;
    end else begin
```

```
        data_delay <= data_in;
    end
end

endmodule
```

### 3.3 仿真验证要求

#### 3.3.1 仿真测试用例

| 测试用例   | 说明         | 验证内容   |
|--------|------------|--------|
| 基本功能测试 | 正常数据流      | 数据正确处理 |
| 边界测试   | 最小 / 最大分辨率 | 极端情况处理 |
| 连续帧测试  | 多帧连续输入     | 状态机正确性 |
| 突发测试   | 连续数据       | 流水线稳定性 |
| 复位测试   | 复位后恢复      | 复位功能   |

#### 3.3.2 仿真波形检查点

```
Verilog
// 测试检查点
initial begin
    // 等待复位完成
    wait(aresetn == 1'b1);
    #100;

    // 检查输出有效
    @(posedge aclk);
    assert(m_axis_tvalid == 1'b1) else $error("Output not valid");

    // 检查数据正确性
    if (m_axis_tdata != expected_value) begin
        $error("Data mismatch at time %0t", $time);
    end
end
```

### 3.4 综合实现要求

#### 3.4.1 综合约束

```
TCL
# 时序约束
create_clock -name aclk -period 8.0 [get_ports aclk]
```

```
set_input_delay -clock aclk -max 2.0 [all_inputs]
set_output_delay -clock aclk -max 2.0 [all_outputs]

# BRAM 约束
set_property RAM_STYLE AUTO [get_cells -hier -filter {NAME =~ *bram*}]

# DSP 约束
set_property USE_DSP48 AUTO [get_cells -hier -filter {NAME =~ *dsp*}]
```

3.4.2 资源目标

| 资源   | 目标利用率 | 说明    |
|------|-------|-------|
| LUT  | ≤ 5%  | 单个 IP |
| FF   | ≤ 5%  | 单个 IP |
| BRAM | ≤ 2%  | 单个 IP |
| DSP  | ≤ 3%  | 单个 IP |

3.5 板级验证要求

3.5.1 验证步骤

- **比特流生成**：成功生成比特流文件
- **下载验证**：通过 JTAG 或 USB 将比特流下载到核心板
- **功能验证**：使用参考设计 Pipeline 测试 IP 功能
- **性能测试**：验证最大分辨率和帧率

3.5.2 验证记录

| 验证项目          | 预期结果 | 实际结果 | 状态 |
|---------------|------|------|----|
| 1080P @ 60fps | 正常输出 | —    | —  |
| 4K @ 30fps    | 正常输出 | —    | —  |
| 4K @ 60fps    | 正常输出 | —    | —  |
| 寄存器配置         | 参数生效 | —    | —  |
| 复位功能          | 正常复位 | —    | —  |

## 四、交付物要求

### 4.1 代码交付物

| 文件      | 说明       | 格式          |
|---------|----------|-------------|
| IP 模块代码 | 核心处理模块   | .v          |
| 仿真 TB   | 功能仿真代码   | _tb.v       |
| 约束文件    | 时序和物理约束  | .sdc / .xdc |
| 寄存器描述   | 寄存器定义头文件 | _regdef.vh  |

### 4.2 文档交付物

| 文档    | 说明         | 建议页数   |
|-------|------------|--------|
| 技术规格书 | IP 功能和技术规格 | 5-10 页 |
| 接口手册  | 输入输出接口定义   | 3-5 页  |
| 使用指南  | 配置和使用说明    | 5-8 页  |
| 测试报告  | 仿真和板级测试结果  | 5-10 页 |

### 4.3 代码目录结构

目录结构

```
[IP名称]/
├── rtl/
│   ├── [ip_name].v           # 顶层模块
│   ├── [ip_name]_core.v      # 核心处理
│   ├── [ip_name]_reg.v       # 寄存器接口
│   └── [ip_name]_fifo.v      # 缓冲模块
├── sim/
│   ├── tb_[ip_name].v        # 顶层 Testbench
│   ├── tb_data.v             # 测试数据
│   └── wave.do                # 波形文件
├── constraints/
│   ├── timing.sdc            # 时序约束
│   └── pinout.pdc            # 引脚约束 (可选)
├── doc/
│   ├── SPEC.md               # 技术规格书
│   └── INTERFACE.md          # 接口手册
```

|

└─ TEST\_REPORT.md

# 测试报告

|

└─ Makefile

# 构建脚本

## 五、评分标准

### 5.1 评分构成

| 评分项   | 权重  | 说明          |
|-------|-----|-------------|
| 代码质量  | 25% | 可读性、规范性、模块化 |
| 功能正确性 | 30% | 仿真通过、板级验证   |
| 性能指标  | 20% | 资源利用率、时钟频率  |
| 文档完整性 | 15% | 规格书、测试报告    |
| 创新性   | 10% | 算法优化、架构创新   |

### 5.2 代码质量评分细则

| 等级  | 分数    | 标准                   |
|-----|-------|----------------------|
| 优秀  | 23-25 | 代码规范、注释完整、模块化良好、易于维护 |
| 良好  | 20-22 | 代码规范、注释较完整、模块化较好     |
| 中等  | 15-19 | 代码基本规范、模块划分合理        |
| 及格  | 10-14 | 代码可运行、结构基本清晰         |
| 不及格 | 0-9   | 代码混乱、难以理解            |

### 5.3 功能正确性评分细则

| 等级 | 分数    | 标准                  |
|----|-------|---------------------|
| 优秀 | 28-30 | 仿真 100% 通过，板级验证全部通过 |
| 良好 | 25-27 | 仿真 95% 通过，板级验证大部分通过 |
| 中等 | 20-24 | 仿真 80% 通过，板级验证基本通过  |

| 等级  | 分数    | 标准              |
|-----|-------|-----------------|
| 及格  | 15-19 | 仿真通过，板级验证部分功能正常 |
| 不及格 | 0-14  | 仿真失败或功能不正确      |

## 六、附录

### 6.1 术语表

| 术语         | 说明                             |
|------------|--------------------------------|
| AXI-Stream | ARM 提出的片上总线协议，用于高速数据传输         |
| ISP        | Image Signal Processor，图像信号处理器 |
| Bayer      | 拜耳阵列，彩色滤波阵列格式                  |
| Gamma      | 伽马校正，显示亮度非线性校正                 |
| AWB        | Auto White Balance，自动白平衡       |
| DPC        | Defect Pixel Correction，坏点校正   |
| BRAM       | Block RAM，FPGA 内嵌块 RAM         |

### 6.2 参考资料

| 资料                                           | 说明           |
|----------------------------------------------|--------------|
| Video-Processing-IP-Roadmap.md               | 视频处理 IP 长期规划 |
| Reference-Design-4K-Video-Capture-Display.md | 4K 视频参考设计    |
| Register-Design-Specification.md             | 寄存器设计规范      |
| IP-Core-Library.md                           | IP 核库文档      |
| Efinity 文档                                   | 易灵思官方开发工具文档  |

### 6.3 技术支持

· 课题邮箱：[待定]

- 技术讨论群：[待定]
- 文档仓库：awesom 项目 Git 仓库

## 附：IP 开发课题列表

以下为第一阶段视频处理 IP 开发课题，每个学生从中选择一项完成开发。

### 课题列表

| 序号 | IP 核名称   | 功能描述             | 难度    | BRAM | 备注                |
|----|----------|------------------|-------|------|-------------------|
| 1  | 色彩空间转换   | RGB、YUV、RAW 格式互转 | ★★☆   | 0 块  | 支持多种格式            |
| 2  | Gamma 校正 | 可配置 Gamma 曲线校正   | ★★☆   | 1 块  | 曲线可配置             |
| 3  | 自动白平衡    | 自动白平衡算法          | ★★★★  | 2 块  | 需统计色温             |
| 4  | 缺陷像素校正   | 坏点检测和校正          | ★★☆   | 0 块  | 静态坏点表             |
| 5  | 降噪 IP    | 空域 / 时域降噪        | ★★★★★ | 4 块  | 3×3 卷积核           |
| 6  | 锐化 IP    | 图像锐化增强           | ★★☆   | 0 块  | Sobel / Laplacian |
| 7  | 对比度增强    | 自动对比度调整          | ★★★★  | 4 块  | 直方图均衡化            |

### 课题详情

#### 课题一：色彩空间转换 IP

功能要求：

- 支持 RGB 到 YUV 转换
- 支持 YUV 到 RGB 转换
- 支持 RAW 到 RGB 转换（去马赛克前置）
- 支持 YUV444 / YUV422 / YUV420 格式转换
- 可配置转换矩阵系数

**输入输出规格：**输入：RGB888 / YUV422 / RAW10/12；输出：RGB888 / YUV422 / RAW10/12；  
最大分辨率：3840 × 2160

算法说明：

```
Verilog
```

```
// RGB to YUV 转换矩阵 (BT.601)
Y = 0.299×R + 0.587×G + 0.114×B
U = -0.169×R - 0.331×G + 0.500×B + 128
V = 0.500×R - 0.419×G - 0.081×B + 128
```

## 课题二：Gamma 校正 IP

### 功能要求：

- 实现可配置的 Gamma 曲线
- 支持预设 Gamma 曲线（1.0 / 2.2 / 2.4 / sRGB）
- 支持自定义 Gamma 曲线查表
- 支持 R/G/B 独立校正

**输入输出规格：**输入：RGB888；输出：RGB888；Gamma 表深度：256

### 算法说明：

算式

$$\text{output} = \text{input}^{(1/\text{gamma})} \times 255$$

## 课题三：自动白平衡 IP

### 功能要求：

- 实现自动白平衡算法（灰度世界假设）
- 支持手动白平衡增益设置
- 支持不同光源模式（日光 / 阴天 / 荧光灯 / 白炽灯）
- 输出白平衡统计信息

**输入输出规格：**输入：RGB888；输出：RGB888；统计窗：可配置

### 算法说明：

算式

$$\begin{aligned} R\_gain &= \text{Avg\_Gray} / \text{Avg\_R} \\ G\_gain &= 1.0 \\ B\_gain &= \text{Avg\_Gray} / \text{Avg\_B} \end{aligned}$$

$$\begin{aligned} R\_out &= R\_in \times R\_gain \\ G\_out &= G\_in \times G\_gain \\ B\_out &= B\_in \times B\_gain \end{aligned}$$

## 课题四：缺陷像素校正 IP

### 功能要求：

- 支持静态坏点表存储

- 支持坏点检测和自动校正
- 支持多种校正算法（邻域平均 / 线性插值）
- 支持坏点使能控制

**输入输出规格：**输入：RAW10/12；输出：RAW10/12；坏点表容量：最大 1024 个

**算法说明：**

伪代码

```
if (pixel == bad_pixel) {  
    output = average(neighbor_pixels);  
}
```

## 课题五：降噪 IP

**功能要求：**

- 实现  $3 \times 3$  空域降噪滤波
- 支持可配置滤波强度
- 支持 BYPASS 模式
- 噪声类型：高斯噪声、椒盐噪声

**输入输出规格：**输入：RGB888；输出：RGB888；延迟： $\leq 1$  行

**算法说明：**

算式

$$\text{output} = \Sigma(\text{kernel}[i] \times \text{neighbor}[i]) / \Sigma(\text{kernel})$$

## 课题六：锐化 IP

**功能要求：**

- 实现多种锐化算子（Sobel / Laplacian / 锐化）
- 支持锐化强度配置
- 支持边缘检测模式
- 支持 BYPASS 模式

**输入输出规格：**输入：RGB888；输出：RGB888；最大分辨率： $3840 \times 2160$

**算法说明：**

算式

```
// Laplacian 锐化  
output = input +  $\alpha \times (5 \times \text{center} - \text{neighbor\_sum})$ 
```

## 课题七：对比度增强 IP

### 功能要求：

- 实现自动对比度增强
- 支持直方图统计和均衡化
- 支持手动对比度调节
- 支持局部对比度增强

**输入输出规格：**输入：RGB888；输出：RGB888；统计窗：可配置

### 算法说明：

伪代码

```
histogram[256] = count(pixel_value)
cdf = cumulative_sum(histogram)
output = cdf[input] × (255 / total_pixels)
```

## 课题选择方式

- 学生根据个人兴趣加入课题微信群
- 注明独自接题，或者小组（不超过 3 人）接题
- 群内将确认课题分配
- 课题确认后如需更换，需提前申请

**文档版本：**V1.0.0 | **最后更新：**2026-04-15 | **课题负责人：**[待定]